MODULE 2 · WEEK 1

Imbalanced Data & Sampling

Fixing class imbalance with under/oversampling, SMOTE & SMOTENC — and cross-validating it without leaking.

Lectures 01–08

This week

1. **Why imbalance breaks models** — accuracy lies when one class is rare
2. **Random under- & oversampling** — the simplest rebalancing levers
3. **SMOTE & SMOTENC** — synthesizing realistic minority examples
4. **Cross-validation** — k-fold, repeated, LOOCV, leave-one-group-out
5. **Doing it right** — resample *inside* CV so you never leak

The problem with imbalanced data

- If 95% of rows are class 0, a model that always says "0" is 95% accurate — and useless.
- The minority class is usually the one you care about (fraud, failure, disease).
- Fixes live on two axes: **change the data** (sampling) or **change the metric** (precision/recall/F1).

Key idea Never judge an imbalanced model on accuracy alone — rebalance the data *and* watch recall on the minority class.

Random undersampling (majority)

- Throw away majority-class rows until the classes are balanced.
- Fast and simple, but you **discard real data** — risky when the dataset is small.

```
from imblearn.under_sampling import RandomUnderSampler
rus = RandomUnderSampler(random_state=42)
X_res, y_res = rus.fit_resample(X_train, y_train)
```

Random oversampling (minority)

- Duplicate minority-class rows until balanced — keeps all majority data.
- Risk: exact copies can let the model **memorize** the minority points (overfit).

```
from imblearn.over_sampling import RandomOverSampler
ros = RandomOverSampler(random_state=42)
X_res, y_res = ros.fit_resample(X_train, y_train)
```

SMOTE — synthetic minority oversampling

- Instead of copying, SMOTE **interpolates** new points between a minority row and its nearest neighbors.
- Gives the model fresh, plausible examples rather than duplicates.
- Plain SMOTE is **numeric-only** — it does math between feature values.

```
from imblearn.over_sampling import SMOTE
sm = SMOTE(k_neighbors=5, random_state=42)
X_res, y_res = sm.fit_resample(X_train, y_train)
```

Key idea SMOTE manufactures believable minority rows along the line between real neighbors — far better than blind duplication.

SMOTENC — mixed numeric + categorical

- Real datasets have categories (state, product, type). Plain SMOTE can't interpolate those.
- **SMOTENC** handles numeric features by interpolation and categorical features by majority vote of the neighbors.
- You just tell it which column indices are categorical.

```
from imblearn.over_sampling import SMOTENC
sm = SMOTENC(categorical_features=[0, 3, 7], random_state=42)
X_res, y_res = sm.fit_resample(X_train, y_train)
```

Cross-validation, part 1

- **k-fold**: split into k folds, train on k−1, test on 1, rotate — every row is tested once.
- **Stratified** k-fold keeps the class ratio in each fold (essential when imbalanced).
- **Repeated** k-fold reruns the whole thing with new shuffles for a more stable estimate.

```
from sklearn.model_selection import RepeatedStratifiedKFold
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3,
                             random_state=42)
```

Cross-validation, part 2

- **LOOCV** (leave-one-out): k = n rows. Maximal data to train, but very slow and high-variance.
- **Leave-one-group-out** (LOGOCV): hold out an entire group each time — e.g. a whole patient, store, or storm.
- Use LOGOCV when rows within a group are correlated and must not be split across train/test.

```
from sklearn.model_selection import LeaveOneGroupOut
logo = LeaveOneGroupOut()
for tr, te in logo.split(X, y, groups=group_id):
    ...
```

Key idea Pick the CV scheme that matches how your data is structured — leaking a group across folds quietly inflates your scores.

SMOTE with ML — the right way

- **Golden rule:** resample *only the training fold*, never the test fold.
- Resampling before the split leaks synthetic neighbors of test rows into training.
- Put SMOTE inside an **imblearn Pipeline** so each CV fold is resampled independently.

```
from imblearn.pipeline import Pipeline
pipe = Pipeline([('smote', SMOTE(random_state=42)),
                 ('clf', RandomForestClassifier())])
scores = cross_val_score(pipe, X, y, cv=cv, scoring='f1')
```

Key idea SMOTE belongs *inside* the pipeline — resample per fold, score on untouched real data.

Key takeaways

- Imbalance makes accuracy meaningless — rebalance and track minority recall / F1.
- Undersample = drop majority; oversample = duplicate minority; **SMOTE** = synthesize minority.
- **SMOTENC** extends SMOTE to mixed numeric + categorical data.
- Match your CV scheme to your data structure (stratified, repeated, LOGOCV).
- Always resample **inside** cross-validation — leakage is the #1 way these projects go wrong.